REDOWAN DELOWAR

# Context propagation in Go

Lessons from operating large-scale distributed systems at DoorDash (Wolt)



Scan for the slides as a PDF

github.com/rednafi/talks

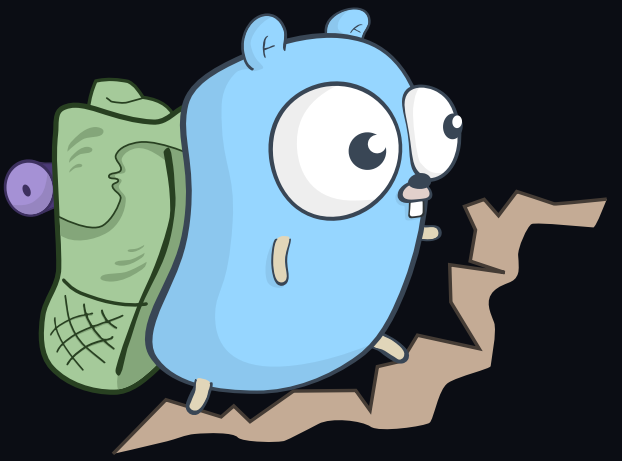
# About me

Hi, I'm Redowan - mostly go by the handle @rednafi on the web.

I work at DoorDash/Wolt where I oscillate between product and platform work.

Interested in databases, streaming, o11y, and failure modes of large-scale distsys.

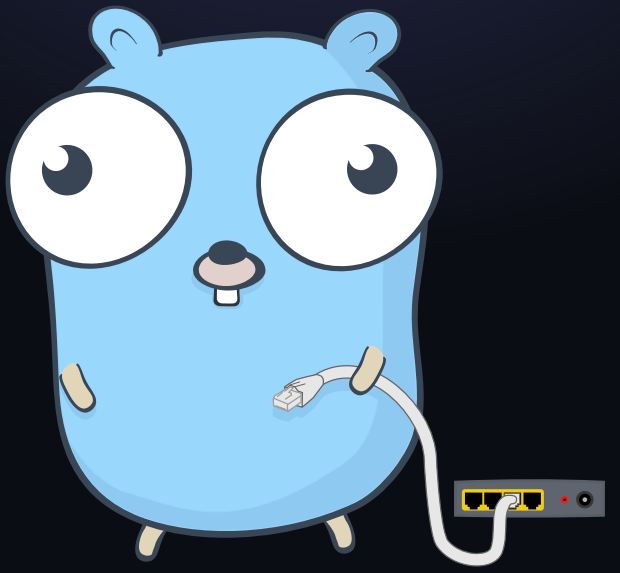
I maintain a blog rednafi.com that gets a few thousand monthly visitors.



github.com/rednafi
linkedin.com/in/rednafi

> A distributed system is one in which the failure of a computer you didn't even know existed can render your own computer unusable.

— LESLIE LAMPORT, 1987



THE PROBLEM

# The log says a context stopped, but not why

order-service

```
ERROR place order: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
ERROR charge card: context canceled
ERROR fetch menu: context deadline exceeded
ERROR place order: context canceled
```

ANY OF THESE CAN CAUSE IT

- The client hung up
- A deadline set by a caller ran out
- The server started to shut down
- Some code called `cancel()`

Often the cancel started in a service or a library I have never read.



> I had a case when on production I got random "context canceled" log messages. And in the case like that you don't even know where to dig and how to investigate it further.

— BULLGARE, GOLANG/GO#51365, 2022



# Overview

01 What a context is, and how code breaks it

02 Passing a context down

03 Recording why a context stopped

04 Context values

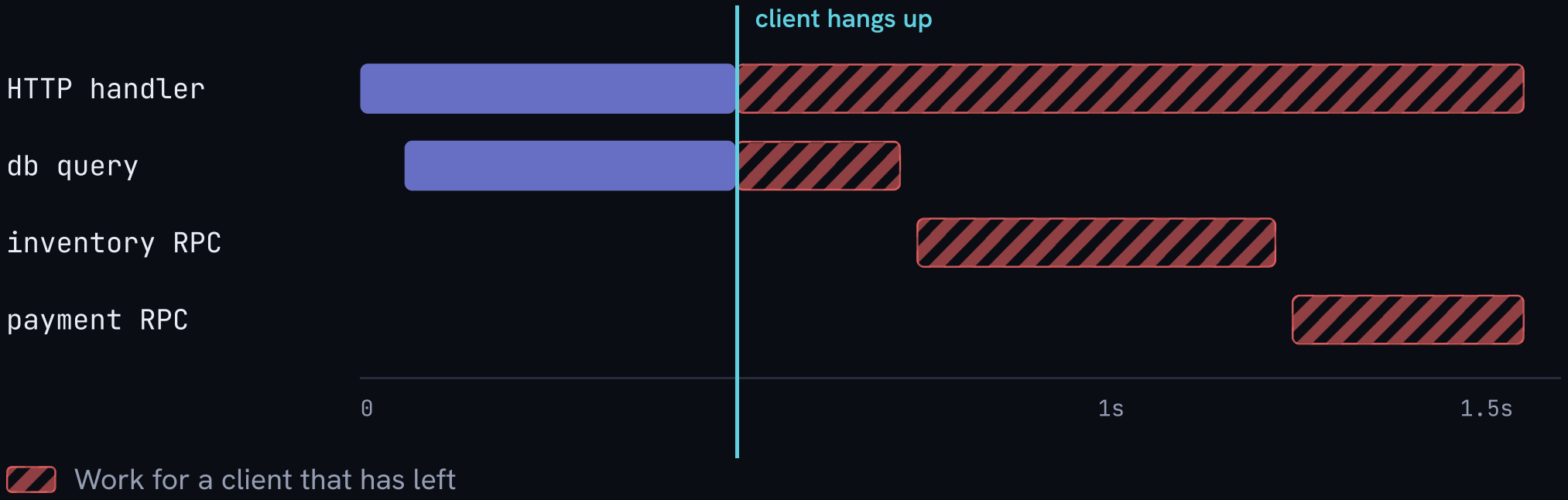
05 Enforcing the rules with linters

01

PART 1

# What's a context?

# When the caller gives up, the work keeps going




The query holds a database connection the whole time

The inventory and payment services do work that nobody reads

# Go can't stop a goroutine from the outside

- A goroutine has no handle and no kill method
- So the work has to be told to stop, and then stop itself
- Every goroutine on the request needs the same signal
- The signal also needs a time limit, so work can stop when time runs out



> When a request is canceled or times out, all the goroutines working on that request should exit quickly so the system can reclaim any resources they are using.

Sameer Ajmani, Go blog, 2014

# context.Context carries the stop signal and the deadline

```
type Context interface {
    Deadline() (deadline time.Time, ok bool)   <- when to give up
    Done() <-chan struct{}                     <- closed when the work should stop
    Err() error                                <- Canceled or DeadlineExceeded
    Value(key any) any                         <- data about the request
}
```

There is no `Cancel` method, so the code that gets the signal can't send it

A context never changes. To add a deadline or a value, you derive a child

> Package context defines the Context type, which carries deadlines, cancellation signals, and other request-scoped values across API boundaries and between processes.

context package doc

# Each With function wraps a parent, so contexts form a tree

`WithCancel(parent)`

Stop when the code calls cancel

`WithTimeout(parent, d)`

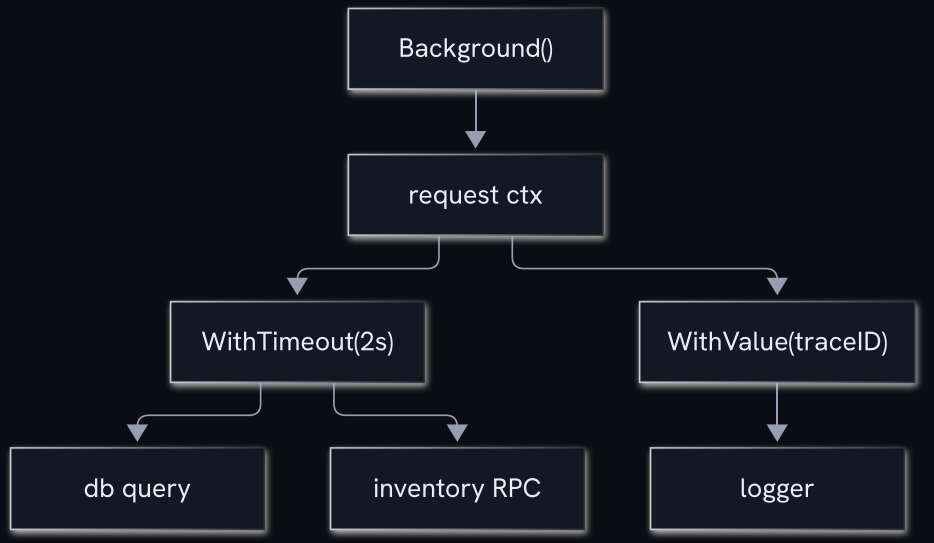
Stop after a duration, or at a time with `WithDeadline`

`WithValue(parent, k, v)`

Add one value

`WithoutCancel(parent)`

Keep the values and drop the stop signal (Go 1.21)




When the 2s timeout fires, the query and the RPC stop. The logger keeps going.

# A context travels inside a process and between processes




In-process: your code passes `ctx` from function to function.
Distributed: gRPC and OpenTelemetry copy it into headers. They can copy only what your code passed down.

# Each layer passes ctx to the next

```go
func (h *Handler) GetOrder(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
    defer cancel()
    order, err := h.orders.Get(ctx, r.PathValue("id"))
    // ...
}

func (s *OrderService) Get(ctx context.Context, id string) (Order, error) {
    return s.store.Find(ctx, id)
}

func (s *OrderStore) Find(ctx context.Context, id string) (Order, error) {
    row := s.db.QueryRowContext(ctx, findOrderSQL, id)
    // ...
}
```

the handler sets the budget

the service passes it on

the driver waits on ctx.Done()

# Canceling a context doesn't stop code that never checks it

```go
// Ignores ctx.
func resizeAll(ctx context.Context, images []Image) error {
    for _, img := range images {
        resize(img) // 200ms each
    }
    return nil
}

// Checks ctx before each image.
func resizeAll(ctx context.Context, images []Image) error {
    for _, img := range images {
        if err := ctx.Err(); err != nil {
            return err
        }
        resize(img)
    }
    return nil
}
```

keeps going after a cancel

stops before the next image

`cancel()` only closes the `Done` channel. It doesn't stop any goroutine

So every loop and every wait has to check `ctx`, or pass it to code that does

# Use a context for deadlines and cancellation first

| | | |
|---|---|---|
| **Deadline** | Stop by this time. Set at the edge and passed to every call. | EVERY REQUEST |
| **Cancellation** | Stop now. The client left, or the process is shutting down. | EVERY REQUEST |
| **Values** | Data about the request that has to cross APIs, such as a trace ID. | SOME REQUESTS |

> Use context Values only for request-scoped data that transits processes and APIs, not for passing optional parameters to functions.

context package doc

# The standard library stops its own work when the context ends

```go
ctx, cancel := context.WithTimeout(ctx, 3*time.Second)
defer cancel()

req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
resp, err := http.DefaultClient.Do(req)

rows, err := db.QueryContext(ctx, listOrdersSQL, userID)

out, err := exec.CommandContext(ctx, "pg_dump", dbName).Output()
```

closes the connection

the driver gets ctx and stops the query

kills the process

The HTTP server cancels `r.Context()` when the client goes away

`signal.NotifyContext` cancels a context when a signal such as `SIGTERM` arrives

# The rules have been written down since 2014

**Jul 2014**

## Go blog

Go Concurrency Patterns: Context. Pass ctx as the first argument.

**Aug 2016**

## Go 1.7

context moves into the standard library.

**Feb 2021**

## Go blog

Contexts and structs. Don't store a context in a struct.

**Nov 2022**

## Google style guide

Rules for contexts and custom contexts, made public.

**2023**

## Go 1.20, 1.21

Causes and WithoutCancel.

The docs explain how to pass a context. They say little about how it breaks, or how to find out why one was canceled.

# Most context bugs I see fall into these groups

## 1

### Work that doesn't stop

*The client left, and the query keeps going.*

A layer drops the context or ignores it.

## 2

### Work that stops too early

*The audit write fails after the response.*

Background work got the request's context.

## 3

### A stop that doesn't say why

*context canceled, with no reason.*

The code that canceled didn't record one.

## 4

### A value that goes missing

*The user ID is gone, and nothing failed to compile.*

A key collided, or a required value hid in the context.

Each piece of advice that follows prevents one of these.

02

PART 2

# Passing a context down

PASSING A CONTEXT DOWN

# Pass ctx as the first argument, all the way down

```
// Don't: a new root in the middle of the chain.
return s.store.Find(context.Background(), id)

// Do: pass the caller's ctx.
return s.store.Find(ctx, id)
```




Use `context.Background()` only in `main` and `init`

In tests, use `t.Context()` (Go 1.24)

Use `context.TODO()` where no context is passed in yet

Never pass a `nil` context

PASSING A CONTEXT DOWN

# Pass ctx to each call instead of storing it in a struct

```go
// Don't: every call uses the context from when the struct was built.
type Notifier struct {
    ctx    context.Context
    client *sms.Client
}


// Do: each caller passes its own.
func (n *Notifier) Send(ctx context.Context, msg string) error {
    return n.client.Send(ctx, msg)
}
```

one deadline for every call

> When you store the context in a struct, you obscure lifetime to the callers, or worse intermingle two scopes together in unpredictable ways.

Go blog, Contexts and structs, 2021

PASSING A CONTEXT DOWN

# Wait on the timer and ctx.Done() together

```
// Don't: the retry sleeps through a cancel.
time.Sleep(backoff(attempt))

// Do: return on whichever comes first.
select {
case <-time.After(backoff(attempt)):
case <-ctx.Done():
    return ctx.Err()
}
```

can't be interrupted

a cancel ends the wait

Every blocking wait needs a `ctx.Done()` case, including channel sends

Since Go 1.23, a `time.After` timer that nobody reads is garbage collected

PASSING A CONTEXT DOWN

# Detach work that has to outlive the request, then bound it

```
// Don't: canceled when the handler returns.
go h.audit.Write(r.Context(), orderID)        <-- fails after the response


// Do: keep the values, drop the cancel, add a bound.
detached := context.WithoutCancel(r.Context())        <-- Go 1.21
h.pool.Submit(func() {
    ctx, cancel := context.WithTimeout(detached, 5*time.Second)   <-- a bound of your own
    defer cancel()
    h.audit.Write(ctx, orderID)
})
```

`context.Background()` also works, and loses the trace ID and the log fields

A pool lets shutdown wait for the work, and a bare `go` doesn't

PASSING A CONTEXT DOWN

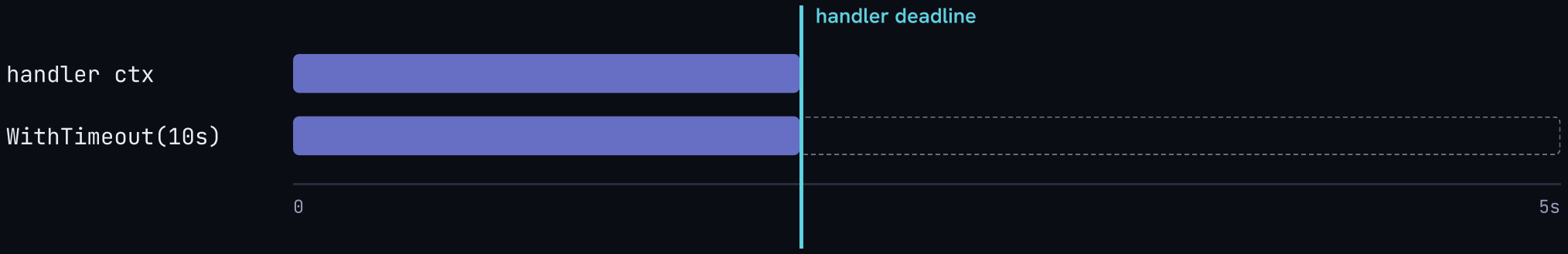
# Set the deadline at the edge, since a child can't extend it

```
// Handler: the request gets 2s.
ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
defer cancel()

// Deeper down: asking for 10s doesn't help.
ctx, cancel := context.WithTimeout(ctx, 10*time.Second)
```

the edge sets the budget

still ends at 2s




The 10s the child asked for, cut at 2s

Give an inner call its own timeout only when it needs less time than the request.

03

PART 3

# Recording why a context stopped

RECORDING WHY A CONTEXT STOPPED

# Cancel with a cause that says why

```go
func processOrder(ctx context.Context, id string) error {
    ctx, cancel := context.WithCancelCause(ctx)
    defer cancel(errors.New("processOrder: completed"))

    if err := checkInventory(ctx, id); err != nil {
        cancel(fmt.Errorf("inventory %s: %w", id, err))
        return err
    }
    // ...
}
```

cancel takes a cause (Go 1.20)

default reason

specific reason

```
$ context.Cause(ctx)
inventory ord-123: connection refused
```

The first cancel wins, and later calls do nothing

`ctx.Err()` still returns `context.Canceled`

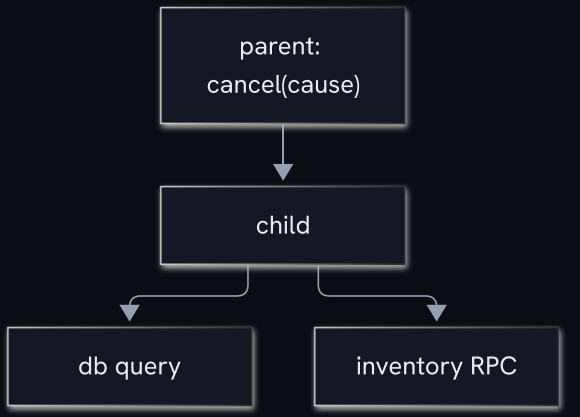
RECORDING WHY A CONTEXT STOPPED

# Read the cause anywhere below the cancel

```go
parent, cancel := context.WithCancelCause(ctx)
child, stop := context.WithTimeout(parent, time.Minute)
defer stop()

cancel(errors.New("order-service: shutting down"))

child.Err()          // context canceled
context.Cause(child) // order-service: shutting down
```




- A function deep in the tree can read a reason set at the top
- The cause stays in the process. gRPC sends only the deadline and a reset

RECORDING WHY A CONTEXT STOPPED

# Don't rely on WithTimeoutCause alone

```
ctx, cancel := context.WithTimeoutCause(ctx, 5*time.Second,
    fmt.Errorf("processOrder %s: 5s timeout", id))
defer cancel()
```

takes no cause

context/context.go

```
return c, func() { c.cancel(true, Canceled, nil) }
```

| PATH | CONTEXT.CAUSE(CTX) |
| --- | --- |
| The timer fires | `processOrder ord-123: 5s timeout` |
| The function returns first | `context canceled` |
| An error path calls `cancel()` | `context canceled` |

RECORDING WHY A CONTEXT STOPPED

# Stack a cancel cause on top of the timeout

```go
func processOrder(ctx context.Context, id string) error {
    ctx, cancelCause := context.WithCancelCause(ctx)
    ctx, cancelTimeout := context.WithTimeoutCause(ctx, 5*time.Second,
        fmt.Errorf("processOrder %s: 5s timeout", id))
    defer cancelTimeout()
    defer cancelCause(errors.New("processOrder: completed"))

    // call cancelCause(...) on each error path
}
```

takes a cause on every path

owns the timer and the deadline

deferred last, so it runs first

`ctx.Err()` is `DeadlineExceeded` when the timer fires

`ctx.Deadline()` still works, so gRPC can send it on

> If users need to set the cause on both the timer and cancel() paths, they can stack contexts.

Sameer Ajmani, golang/go#56661, 2022

RECORDING WHY A CONTEXT STOPPED

# Wrap the sentinel in every cause, so errors.Is still matches

```
ctx, cancel := context.WithTimeoutCause(ctx, 20*time.Millisecond,
    errors.New("inventory: budget spent"))   <-- no %w, so the sentinel is lost
defer cancel()

req, _ := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
_, err := http.DefaultClient.Do(req)

errors.Is(err, context.DeadlineExceeded)   <-- false
```

Since Go 1.23, `net/http` returns `context.Cause(ctx)`, so a plain cause takes the place of the sentinel

Retry and alert code that checks `DeadlineExceeded` stops matching

```
fmt.Errorf("inventory: budget spent: %w", context.DeadlineExceeded)   <-- wrapped with %w, so true
```

RECORDING WHY A CONTEXT STOPPED

# Log the error and the cause as separate fields

```go
if err := ctx.Err(); err != nil {
    logger.LogAttrs(ctx, slog.LevelError, "request failed",
        slog.Any("err", err),
        slog.Any("cause", context.Cause(ctx)),
    )
}
```

```
level=ERROR msg="request failed"
    err="context deadline exceeded"
    cause="processOrder ord-123: 5s timeout: context deadline exceeded"
```

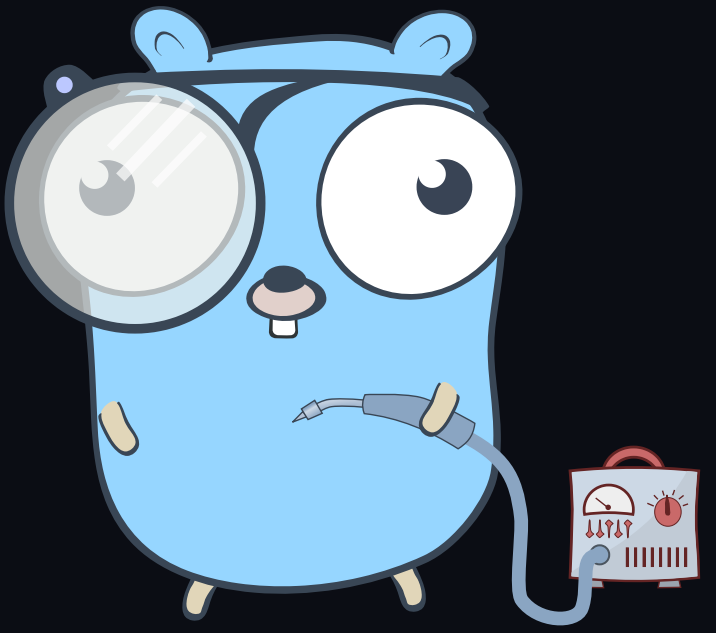
`err` gives the kind, so dashboards can group by it

`cause` gives the reason, so on-call knows what to do

RECORDING WHY A CONTEXT STOPPED

# Give every cancel in your code a cause

- The `net/http` server cancels `r.Context()` with no cause when the client hangs up
- Most client libraries were written before Go 1.20
- `os/signal.NotifyContext` got a cause only in Go 1.26
- If every cancel in your code has a cause, a bare `context canceled` came from outside your code



04

PART 4

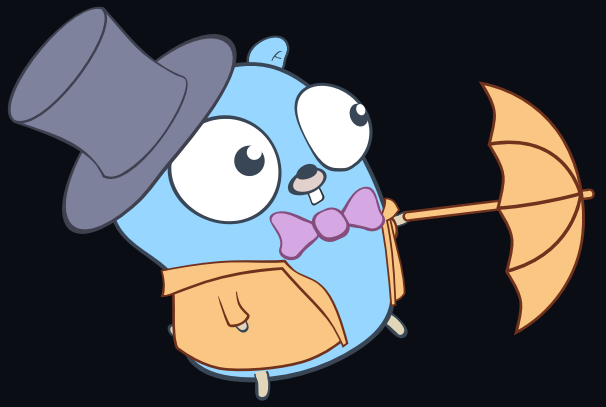
# Context values

CONTEXT VALUES

# Put a value in the context only if the code can run without it

## FINE IN A CONTEXT

- Trace, request, and correlation IDs
- Auth data set by middleware
- Request-scoped log fields
- Idempotency keys



## MAKE IT A PARAMETER

- User data the function works on
- Database handles and service clients
- Configuration
- Business inputs

> Use context Values only for request-scoped data that transits processes and APIs, not for passing optional parameters to functions.
>
> context package doc

CONTEXT VALUES

# Let middleware pass request data to the handler in the context

```
func (a *Auth) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        user, err := a.sessions.User(r.Context(), r.Header.Get("Authorization"))
        // ...
        ctx := auth.WithUser(r.Context(), user)
        next.ServeHTTP(w, r.WithContext(ctx))
    })
}
```




The middleware already did the lookup, so the handler doesn’t repeat it

The handler reads the user once and passes it on as an argument

# Use an unexported key type, so keys can't collide

```
// Don't: two packages pick the same string.
ctx = context.WithValue(ctx, "user", "alice")     // package auth
ctx = context.WithValue(ctx, "user", "audit-bot") // package audit
ctx.Value("user")                                 // "audit-bot"  ← alice is gone


// Do: a key type that only this package can name.
package auth

type ctxKey struct{}  ← no other package can build one
```

`Value` walks up the tree and returns the first key that is equal

Two string keys with the same text are equal

CONTEXT VALUES

# Hide the key behind a setter and a getter

```
package auth

func WithUser(ctx context.Context, u User) context.Context {
    return context.WithValue(ctx, ctxKey{}, u)
}

func UserFromContext(ctx context.Context) (User, bool) {
    u, ok := ctx.Value(ctxKey{}).(User)
    return u, ok
}
```

← the only way in

← the one type assertion

Callers get a typed value and a bool, and never touch the key

A missing value gives `false`, not a panic

CONTEXT VALUES

# The standard library and OpenTelemetry also hide their keys

| PACKAGE | SET | GET |
| --- | --- | --- |
| `net/http/httptrace` | `WithClientTrace` | `ContextClientTrace` |
| `runtime/pprof` | `WithLabels` | `Label` |
| `otel/trace` | `ContextWithSpan` | `SpanFromContext` |
| `grpc/metadata` | `NewOutgoingContext` | `FromIncomingContext` |

Each package exports a setter and a getter, and keeps the key type unexported.

CONTEXT VALUES

# Don't hide a dependency in a context

```go
// Don't do this.
func (s *BookStore) Create(ctx context.Context, b Book) (int64, error) {
    var db DBTX = s.db
    if tx, ok := TxFromContext(ctx); ok {
        db = tx
    }
    // ...
}


// Do this: the transaction is an argument.
err := store.Tx(ctx, func(tx book.Store) error {
    // use tx for every write
})
```

forget WithTx and this is false, with no error

The writes go to the pool and are no longer atomic

The compiler can’t help, because the dependency is invisible

05

PART 5

# Enforcing the rules

ENFORCING THE RULES

# Let linters catch what code review misses

| | |
|---|---|
| `go vet: lostcancel` | A cancel function that is never called |
| `staticcheck: SA1012` | A `nil` context passed to a function |
| `staticcheck: SA1029` | A built-in type, such as a string, used as a key |
| `containedctx` | A context stored in a struct field |
| `contextcheck` | A function that uses a context it didn't get from its caller |
| `errorlint` | `err == context.Canceled`, which fails on a wrapped cause |

ENFORCING THE RULES

# Make every cancel take a cause

```
package xctx

func WithCancel(parent context.Context) (context.Context, context.CancelCauseFunc) {
    ctx, cancel := context.WithCancelCause(parent)
    return ctx, func(cause error) {
        cancel(fmt.Errorf("%w: %w", cause, context.Canceled))
    }
}


# .golangci.yml
linters:
  settings:
    forbidigo:
      forbid:
        - pattern: ^context\.With(Cancel|Timeout|Deadline)$
          msg: use xctx, which takes a cause
```

the cancel func needs a reason

errors.Is still matches

At DoorDash, Alex Hornbake built this as `xctx` for a Go monorepo that many teams share.

WRAP-UP

# The checklist I use in code review

1. Pass `ctx` first, all the way down, and don't store it in a struct
2. Wait on `ctx.Done()` in every loop and every blocking wait
3. Detach with `WithoutCancel`, then add your own timeout
4. Give every cancel a cause, including the normal return
5. Make every cause wrap `context.Canceled` or `context.DeadlineExceeded`
6. Log `ctx.Err()` and `context.Cause(ctx)` as separate fields
7. Keep values optional and keys unexported, behind a setter and a getter
8. Let linters check the rest

REFERENCES

# Further reading

## GO BLOG

- Go Concurrency Patterns: Context, Sameer Ajmani, 2014
- Go Concurrency Patterns: Pipelines and cancellation, Sameer Ajmani, 2014
- Contexts and structs, Jean Barkhuysen and Matt T. Proud, 2021

## MY POSTS

- What canceled my Go context?
- Avoiding collisions in Go context keys
- What belongs in Go's context values?
- Is passing user ID through context an antipattern?

## GO ISSUES

- #26356: context: ease debugging of where a context was canceled?
- #40221: context: add WithoutCancel
- #51365: context: add APIs for writing and reading cancelation cause
- #56661: context: add APIs for setting a cancelation cause when deadline or timer expires

## DOCS AND STYLE GUIDES

- Package context
- Go 1.20 release notes: context
- Go 1.21 release notes: context
- Google Go style guide: Contexts
- Google Go style guide: Custom contexts

## LINTERS

- go vet: lostcancel
- staticcheck: SA1012, SA1029
- golangci-lint: containedctx, contextcheck, errorlint, forbidigo

# Thanks

Questions?

REDNAFI.COM

